

FC7xxx OCU Integration Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/10/20	Initial release for MCAL V0.3.0

Table of Contents

Revision History	2
Table of Contents	3
Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area	9
Chapter 5 Interrupt Service Routine (ISR)	10
Chapter 6 Error Report	11
6.1 Det	11
6.2 Dem	11
Chapter 7 Function Calls	12
7.1 Function Calls during Startup	12
7.2 Function Calls during Shutdown	12
7.3 Function Calls during Wake-up	12
7.4 Function Calls during Runtime	12
Chapter 8 Other Requirements	13
8.1 Notification, Callback, Callout	13
8.2 Macros	13
Chapter 9 Integration Steps	14

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the OCU module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for OCU module.
- Port: This module provides the port configurations if the user wants to output OCU signal through pin.
- Common: This module is the basic module which used to choose the chip.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.

Module initialization dependency

- Mcu: To use the OCU module, the user needs to initialize MCU module first.
- Port: To output OCU signal, the user needs to initialize PORT module first.

2.2 Files Required for Compile

OCU module files:

- _MCAL/Src/Ocu/src/Ocu.c
- _MCAL/Src/Ocu/src/Ocu_Ftu.c
- _MCAL/Src/Ocu/src/Ocu_Hw.c
- _MCAL/Src/Ocu/include/Ocu.h
- _MCAL/Src/Ocu/include/Ocu_Ftu.h
- _MCAL/Src/Ocu/include/Ocu_Ftu_RegOps.h
- _MCAL/Src/Ocu/include/Ocu_Ftu_Types.h
- _MCAL/Src/Ocu/include/Ocu_Hw.h
- _MCAL/Src/Ocu/include/Ocu_Hw_Irq.h
- _MCAL/Src/Ocu/include/Ocu_Hw_Types.h
- _MCAL/Src/Ocu/include/Ocu_Irq.h
- _MCAL/Src/Ocu/include/Ocu_MemMap.h
- _MCAL/Src/Ocu/include/Ocu_Types.h
- _MCAL/Src/Ocu/include/Ocu_version.h
- _MCAL_generate/src/Ocu_PBcfg.c
- _MCAL_generate/include/Ocu_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- MCAL/Src/Common/include/Platform_Types.h

- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h
- _MCAL/Src/Common/src/Ftu_Common.c

Det module files:

- Det.h

Ret module files:

- SchM_Ocu.h

2.3 Add Plug-ins

OCU module plug-ins are developed for EB tresos Studio, so, to use OCU plug-ins on the EB tresos Studio, the user needs to the OCU module to the EB plug-ins folder first.

- 1) Copy the OCU module(_MCAL/EB_Plugins/eclipse/plugins/OCU) folder to EB tresos plugins (EB/tresos/plugins/) folder.
- 2) Set OCU module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure OCU module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
OCU_START_SEC_CONFIG_DATA_8 OCU_STOP_SEC_CONFIG_DATA_8 OCU_START_SEC_CONFIG_DATA_16 OCU_STOP_SEC_CONFIG_DATA_16 OCU_START_SEC_CONFIG_DATA_32 OCU_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
OCU_START_SEC_CONFIG_DATA_UNSPECIFIED OCU_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
OCU_START_SEC_CONST_BOOLEAN OCU_STOP_SEC_CONST_BOOLEAN OCU_START_SEC_CONST_8 OCU_STOP_SEC_CONST_8 OCU_START_SEC_CONST_16 OCU_STOP_SEC_CONST_16 OCU_START_SEC_CONST_32 OCU_STOP_SEC_CONST_32 OCU_START_SEC_CONST_UNSPECIFIED OCU_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
OCU_START_SEC_CODE OCU_STOP_SEC_CODE OCU_START_SEC_RAMCODE OCU_STOP_SEC_RAMCODE OCU_START_SEC_CODE_AC OCU_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
OCU_START_SEC_VAR_NO_INIT_BOOLEAN OCU_STOP_SEC_VAR_NO_INIT_BOOLEAN OCU_START_SEC_VAR_NO_INIT_8 OCU_STOP_SEC_VAR_NO_INIT_8 OCU_START_SEC_VAR_NO_INIT_16 OCU_STOP_SEC_VAR_NO_INIT_16 OCU_START_SEC_VAR_NO_INIT_32 OCU_STOP_SEC_VAR_NO_INIT_32 OCU_START_SEC_VAR_NO_INIT_UNSPECIFIED OCU_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
OCU_START_SEC_VAR_INIT_BOOLEAN OCU_STOP_SEC_VAR_INIT_BOOLEAN OCU_START_SEC_VAR_INIT_8 OCU_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
OCU_START_SEC_VAR_INIT_16 OCU_STOP_SEC_VAR_INIT_16 OCU_START_SEC_VAR_INIT_32 OCU_STOP_SEC_VAR_INIT_32 OCU_START_SEC_VAR_INIT_UNSPECIFIED OCU_STOP_SEC_VAR_INIT_UNSPECIFIED		startup code (data).

Chapter 4 Exclusive Area

OCU module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the OCU driver:

- Ocu.c
 - Ocu_SetBitChState: exclusive area 0
 - Ocu_ClearBitChState: exclusive area 1
 - Ocu_ClearBitChState: exclusive area 2

- Ocu_Ftu.c:
 - Ocu_Ftu_StartChannel : exclusive area 8
 - Ocu_Ftu_StopChannel : exclusive area 9
 - Ocu_Ftu_SetPinAction: exclusive area 10
 - Ocu_Ftu_SetPinState : exclusive area 11
 - Ocu_Ftu_SetRelativeThreshold: exclusive area 12
 - Ocu_Ftu_SetAbsoluteThreshold : exclusive area 13
 - Ocu_Ftu_ProcessCommonInterrupt : exclusive area 15

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
FTU0	FTU0_IRQHandler	116
FTU1	FTU1_IRQHandler	117
FTU2	FTU2_IRQHandler	118
FTU3	FTU3_IRQHandler	119
FTU4	FTU4_IRQHandler	120
FTU5	FTU5_IRQHandler	121
FTU6	FTU6_IRQHandler	122
FTU7	FTU7_IRQHandler	123
FTU8	FTU8_IRQHandler	124
FTU9	FTU9_IRQHandler	125
FTU10	FTU10_IRQHandler	126
FTU11	FTU11_IRQHandler	127

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Ocu_Init	OCU_E_ALREADY_INITIALIZED; OCU_E_INIT_FAILED
Ocu_DeInit	OCU_E_UNINIT; OCU_E_PARAM_INVALID_STATE
Ocu_StartChannel	OCU_E_UNINIT; OCU_E_PARAM_INVALID_CHANNEL; OCU_E_BUSY
Ocu_StopChannel	OCU_E_UNINIT; OCU_E_PARAM_INVALID_CHANNEL
Ocu_SetPinState	OCU_E_UNINIT; OCU_E_PARAM_INVALID_CHANNEL; OCU_E_PARAM_NO_PIN; OCU_E_PARAM_INVALID_STATE
Ocu_SetPinAction	OCU_E_UNINIT; OCU_E_PARAM_INVALID_CHANNEL; OCU_E_PARAM_NO_PIN; OCU_E_PARAM_INVALID_ACTION
Ocu_GetCounter	OCU_E_UNINIT; OCU_E_PARAM_INVALID_CHANNEL
Ocu_SetAbsoluteThreshold	OCU_E_UNINIT; OCU_E_PARAM_INVALID_CHANNEL; OCU_E_PARAM_INVALID_VALUE
Ocu_SetRelativeThreshold	OCU_E_UNINIT; OCU_E_PARAM_INVALID_CHANNEL
Ocu_EnableNotification	OCU_E_UNINIT; OCU_E_PARAM_INVALID_CHANNEL; OCU_E_NO_VALID_NOTIF
Ocu_DisableNotification	OCU_E_UNINIT; OCU_E_PARAM_INVALID_CHANNEL; OCU_E_NO_VALID_NOTIF
Ocu_GetVersionInfo	OCU_E_PARAM_POINTER

6.2 Dem

None

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API needs be called is OCU_Init(ConfigPtr).

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

If the user configures notifications and the value is not "NULL_PTR" or "NULL", an extern declaration will be generated in Ocu_PBCfg.c. The syntax of this function is as follows: void Ocu_Notification_#channel(void). An extern declaration of the notification functions is available in Ocu_PBCfg.c. The notification functions have to be implemented by the user.

- Callback

None

- Callout

None

8.2 Macros

Please have a look in various definitions available in Common module include file Mcal.h for details.

- If OS not used:

AUTOSAR_OS_NOT_USED needs to be defined.

In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example:

```
extern ISR(OCU_FTU_0_ISR);
void FTU0_IRQHandler(void)
{
    OCU_FTU_0_ISR();
}
```

- If OS is used:

Do not define AUTOSAR_OS_NOT_USED

Chapter 9 Integration Steps

- 1) Configure the OCU module and generate configuration files (Please refer to Building chapter for details)
- 2) Configure appropriate memory sections in linker file or other (Please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations. (Please refer to chapter ISR for details).
- 4) Build the OCU module with dependent modules.